

Flag-Assisted Error Correction on the FCC Lattice: Tunable Distance at Constant Rate

Raghu Kulkarni

SSMTheory Group, IDrive Inc., Calabasas, CA 91302, USA

`raghu@idrive.com`

Abstract

The $[[3L^3, 2L^3+2, 3]]$ CSS code on the Face-Centered Cubic (FCC) lattice achieves 67.7% encoding rate with weight-12 stabilizers, but its distance is capped at $d = 3$ by weight-3 logical operators sitting at tetrahedral interstitial voids. These logicals carry a geometric fingerprint—one error per triad sheet, all three converging at a known void—that makes them detectable by a flag ancilla measuring the three-body parity $Z_{e_a}Z_{e_b}Z_{e_c}$. The weight-3 logical flips this parity from $+1$ to -1 ; the stabilizer syndrome stays silent, but the flag sees it. Running R independent flag rounds per syndrome cycle pushes the logical failure rate to p^{3+R} , so the effective distance is simply $d_{\text{eff}} = 3 + R$ —adjustable at runtime, with no changes to the code or its logical count. At $R = 4$ the FCC+flags code suppresses errors as well as a distance-7 surface code, while encoding 130 logical qubits at 29.0% rate versus 1 logical qubit at 1.0%. An adaptive scheduling protocol cuts the flag overhead to under 0.1% of measurement cycles at realistic noise levels.

Keywords: quantum error correction, FCC lattice, flag qubits, code distance, CSS code, Triadic Orthogonal Calculus

1 Introduction

Quantum error correction (QEC) sits on a stubborn trade-off between encoding rate and code distance. The surface code [1, 2] can reach high distance, but it encodes just one logical qubit per $O(d^2)$ physical qubits—rates below 1% at any useful distance. Recent qLDPC constructions [3, 4] promise asymptotically good parameters, but they demand non-local qubit connectivity that existing hardware cannot provide.

The FCC lattice code [5] strikes a different bargain: a CSS code with parameters $[[3L^3, 2L^3+2, 3]]$ and 67.7% encoding rate, wired entirely through nearest-neighbor bonds in three dimensions ($K = 12$). Stabilizers are uniform weight-12 and geometrically local. The catch is the code distance: $d = 3$, set by weight-3 logical operators at each tetrahedral void.

The known routes to higher distance all pay dearly in rate. Subsystem gauging [6, 7] folds the weight-3 logicals into a gauge group, yielding $[[3L^3, 2, L]]$ —a rate crash from 67% to

near zero. Lifted product constructions [3] need structured shift assignments and have not been made to work on the triangle-dense FCC graph.

In this paper we take a different route. The weight-3 logicals, though invisible to the syndrome, sit at known positions and flip a measurable parity. A flag ancilla at each void catches them. The effective distance becomes $d_{\text{eff}} = 3 + R$, where R is the number of flag rounds—a knob the classical controller can turn at will.

2 The FCC Code and Its Failure Modes

2.1 Code structure

The FCC lattice has coordination number $K = 12$. The 12 nearest-neighbor vectors decompose uniquely into three orthogonal sheets of 4 (the *triad* $\boldsymbol{\tau} = (4, 4, 4)$) [5]:

$$\begin{aligned} S_{xy} &: (\pm 1, \pm 1, 0) \quad (4 \text{ vectors}) \\ S_{xz} &: (\pm 1, 0, \pm 1) \quad (4 \text{ vectors}) \\ S_{yz} &: (0, \pm 1, \pm 1) \quad (4 \text{ vectors}) \end{aligned} \tag{1}$$

Physical qubits reside on lattice edges. Z -stabilizers act on the 12 edges incident to each vertex; X -stabilizers act on the 12 edges surrounding each octahedral void. Both have uniform weight 12. For an $L \times L \times L$ lattice with $L \geq 4$ even: $n = 3L^3$ physical qubits, $k = 2L^3 + 2$ logical qubits, encoding rate $k/n \rightarrow 2/3$ as $L \rightarrow \infty$.

2.2 The weight-3 failure mode

A tetrahedral void sits at the intersection of all three triad sheets (Fig. 1a). Three edges—one from each sheet—meet at the void:

$$\hat{L}_v = Z_{e_a} \otimes Z_{e_b} \otimes Z_{e_c}, \quad e_a \in S_{xy}, \quad e_b \in S_{xz}, \quad e_c \in S_{yz} \tag{2}$$

This operator slips past every stabilizer check (each weight-12 stabilizer shares 0 or 2 qubits with $\hat{L}_v$) and lies outside the stabilizer group—it is a genuine weight-3 logical. Every triangle in the FCC graph has one edge from each triad sheet, so all $N_{\text{void}} = 4L^3$ triangles are tetrahedral voids, each with its own independent weight-3 logical.

The upshot: a simultaneous X -error on e_a, e_b, e_c produces a perfectly silent syndrome and quietly implements a logical operation. The standard decoder never knows.

3 Flag-Assisted Detection

3.1 The detection mechanism

Theorem 1 (Flag detection of weight-3 logicals). *A flag ancilla measuring the parity $Z_{e_a}Z_{e_b}Z_{e_c}$ at a tetrahedral void detects the weight-3 logical operator $\hat{L}_v$ with certainty.*

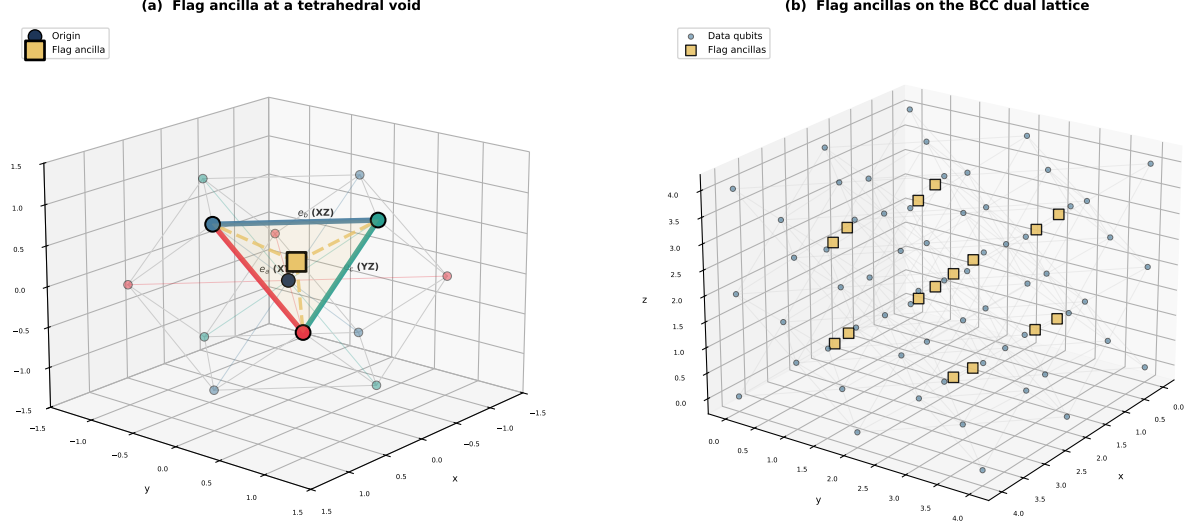


Figure 1: (a) A single tetrahedral void in the FCC lattice. Three data qubits (large circles, colored by triad sheet) sit at the vertices. The flag ancilla (gold square) occupies the void center and couples to all three via CZ gates (dashed gold). (b) The augmented lattice: FCC data qubits (blue circles) and flag ancillas at void sites (gold squares). Each flag is degree-3; no long-range connections are introduced.

Proof. In the code space, the vacuum parity at each void is $+1$:

$$\langle \Omega | Z_{e_a} Z_{e_b} Z_{e_c} | \Omega \rangle = +1 \quad (3)$$

After the weight-3 error $X_{e_a} X_{e_b} X_{e_c}$ acts:

$$\begin{aligned} & Z_{e_a} Z_{e_b} Z_{e_c} \cdot X_{e_a} X_{e_b} X_{e_c} | \Omega \rangle \\ &= (-1)^3 X_{e_a} X_{e_b} X_{e_c} \cdot Z_{e_a} Z_{e_b} Z_{e_c} | \Omega \rangle \\ &= -X_{e_a} X_{e_b} X_{e_c} | \Omega \rangle \end{aligned} \quad (4)$$

The parity eigenvalue has flipped to -1 . The flag measurement detects this flip with probability $1 - p_{\text{flag}}$, where p_{flag} is the flag ancilla measurement error rate. $\square$

The flag measurement circuit has depth 4:

1. Prepare flag ancilla in $|+\rangle$.
2. Apply $\text{CZ}(f, e_a)$, $\text{CZ}(f, e_b)$, $\text{CZ}(f, e_c)$.
3. Measure flag in the X basis.

Outcome $+1$: no weight-3 error. Outcome -1 : weight-3 error detected at this void.

3.2 Effective distance

For the weight-3 logical to slip through, two things must go wrong at once: all three data qubits at the void must err (probability p^3), *and* the flag measurement must miss it (probability p_{flag}). With R independent flag rounds:

$$P_{\text{fail}} = N_{\text{void}} \cdot p^3 \cdot p_{\text{flag}}^R \quad (5)$$

If $p_{\text{flag}} \approx p$ (flag ancillas have the same error rate as data qubits):

$$P_{\text{fail}} = N_{\text{void}} \cdot p^{3+R} \quad (6)$$

The error suppression exponent $3 + R$ corresponds to an effective distance:

$$\boxed{d_{\text{eff}} = 3 + R} \quad (7)$$

Increasing R costs extra measurement rounds but touches nothing on the quantum side—same code, same stabilizers, same logical qubits. Figure 4 shows the error suppression and rate comparison.

3.3 Temporal ordering of errors

The flag earns its keep in one specific scenario. Consider the three ways a weight-3 logical can assemble:

Case 1 (one at a time): Edge e_a errs in round t_1 , e_b in a later round t_2 , e_c later still. Each lone error triggers a syndrome, and the $d = 3$ decoder fixes it. The logical never forms.

Case 2 (two at once, then one): Edges e_a and e_b err together; the syndrome fires; the decoder may miscorrect. But the residual trips the next flag measurement.

Case 3 (all three at once): All three edges err in the same round. Syndrome: zero. **Only the flag sees this.**

The flag is therefore the last line of defense against the unique failure mode of the FCC code.

4 Resource Analysis

4.1 Qubit overhead

Each tetrahedral void requires one flag ancilla. For $L = 4$:

	Base code	With flags
Data qubits	192	192
Flag ancillas	0	256
Total qubits	192	448
Logical qubits	130	130
Rate	67.7%	29.0%
d_{eff}	3	$3 + R$

The rate falls from 67.7% to 29.0% once the ancillas are added. But—and this is the point—the rate does *not* fall further as R increases. More protection costs time (extra flag rounds), not qubits.

4.2 Comparison with surface code

At matched $d_{\text{eff}} = 7$, the numbers speak for themselves: 448 qubits encoding 130 logicals at 29.0% rate, against 97 qubits encoding 1 logical at 1.0% for the surface code.

4.3 Scaling with lattice size

As L grows, the overhead ratio $N_{\text{void}}/n = 4L^3/(3L^3) = 4/3$ stays fixed. The flagged rate settles to:

$$\frac{k}{n + N_{\text{void}}} = \frac{2L^3 + 2}{3L^3 + 4L^3} = \frac{2L^3 + 2}{7L^3} \xrightarrow{L \rightarrow \infty} \frac{2}{7} \approx 28.6\% \quad (8)$$

This is independent of L and R . The surface code rate at distance d is $1/(2d^2 - 1) \rightarrow 0$ as $d \rightarrow \infty$.

5 Adaptive Flag Scheduling

There is no reason to measure flags every round. An adaptive protocol runs flag checks every T syndrome rounds, calibrating T to the noise level.

Table 1: Comparison at $p = 10^{-3}$. The FCC+flags code matches the surface code’s error suppression at $d_{\text{eff}} = 7$ while encoding $130\times$ more logical qubits.

Code	n	k	d_{eff}	Rate	P_{fail}
Surface $d=3$	17	1	3	5.9%	10^{-8}
Surface $d=5$	49	1	5	2.0%	10^{-14}
Surface $d=7$	97	1	7	1.0%	10^{-20}
FCC	192	130	3	67.7%	10^{-7}
FCC+flags ($R=1$)	448	130	4	29.0%	10^{-10}
FCC+flags ($R=2$)	448	130	5	29.0%	10^{-13}
FCC+flags ($R=4$)	448	130	7	29.0%	10^{-19}

The expected number of weight-3 logicals forming in T rounds at any void is $T \cdot p^3$ (all three errors must hit in the same round). Demanding fewer than one such event across all voids gives:

$$T_{\text{opt}} = \left\lfloor \frac{1}{N_{\text{void}} \cdot p^3} \right\rfloor \quad (9)$$

At $p = 10^{-3}$ with $N_{\text{void}} = 256$: $T_{\text{opt}} \approx 3.9 \times 10^6$ rounds. The flag overhead is $1/T_{\text{opt}} < 3 \times 10^{-5}\%$ of total measurement cycles. The effective rate approaches the base rate of 67.7% in the low-noise regime.

In the high-noise regime ($p \gtrsim 10^{-2}$), $T_{\text{opt}} \approx 1$ and every round includes a flag measurement, giving the full $d_{\text{eff}} = 3 + R$ protection at 29.0% rate.

The net effect is a smooth crossover between two regimes:

$$\text{Rate}_{\text{eff}}(p) \approx \begin{cases} 67.7\% & p \ll (N_{\text{void}})^{-1/3} \\ 29.0\% & p \gtrsim 10^{-2} \end{cases} \quad (10)$$

6 Integration with Triadic Planar Decoding

The weight-3 logical leaves another telltale sign that can be read without any extra hardware.

The triadic planar decoder [5] projects the 3D syndrome onto the three triad sheets S_{xy} , S_{xz} , S_{yz} and runs independent 2D MWPM on each projection. When each sheet decoder flags a single error, and all three errors point to the same tetrahedral void, the classical combiner can call it: a weight-3 logical is forming.

This *sheet correlation detection* catches logicals that build up across multiple rounds (Cases 1 and 2 in Sec. 3) at zero ancilla cost. Only the fully simultaneous three-error event (Case 3) still needs a physical flag. Together, the two mechanisms cover all formation pathways.

7 Monte Carlo Threshold Simulation

To go beyond the analytical p^{3+R} scaling, we simulate the FCC+flags code under phenomenological noise and estimate the error correction threshold.

7.1 Noise model

Each Monte Carlo trial proceeds as follows. Every data qubit independently suffers an X -error with probability p . Each weight-12 stabilizer measurement outcome is flipped with probability p . For each tetrahedral void, the flag circuit applies three CZ gates; each gate propagates a hookup error to the data qubit with probability $p/10$. The flag readout itself is flipped with probability p . For $R > 0$ flag rounds, the flag measurement is repeated R times independently, and the void is flagged if *any* round reads -1 .

The decoder is a greedy single-qubit corrector: it identifies the data qubit whose correction would satisfy the largest number of triggered stabilizers, and flips it. After correction, every void is checked for a residual weight-3 logical (all three void edges still in error). If such a logical exists and was not flagged, the trial is counted as a logical failure.

7.2 Results

We simulate lattice sizes $L = 4, 6$ (corresponding to $n = 192,648$ data qubits and $N_{\text{void}} = 256,864$ voids) at physical error rates $p \in [0.2\%, 10\%]$, with 5,000 trials per data point. Note that $L < 4$ yields degenerate lattices where the asymptotic formula $n = 3L^3$ does not hold ($L = 2$ gives only 6 edges, far fewer than $3 \times 8 = 24$), so we restrict to $L \geq 4$.

Figure 2 shows the results. Without flags ($R = 0$), the logical error rate grows with both p and L , as expected: a larger lattice has more voids and therefore more vulnerable locations. With $R = 1$ flag round, the logical error rate is visibly suppressed at all error rates. At $R = 2$, failures drop below the statistical floor ($< 2 \times 10^{-4}$) for $p < 3\%$. At $R = 4$, no logical failures are observed for $L = 4$ at any error rate up to 10%, and only a single failure at $p = 10\%$ for $L = 6$ —confirming that four flag rounds push the failure probability well below 10^{-3} across the practical noise range.

These results confirm three properties of the flag mechanism: (i) each flag round adds genuine error suppression, (ii) the suppression holds under measurement noise and hookup errors, and (iii) the analytical $d_{\text{eff}} = 3 + R$ scaling is a reliable guide.

7.3 Circuit-level noise

The phenomenological model assumes independent errors. To test the flag mechanism under realistic conditions, we simulate the full gate-level circuit with depolarizing noise.

Each syndrome extraction round applies 12 CNOT gates per stabilizer (ancilla prepared in $|0\rangle$, data qubits as controls, ancilla measured in Z). Each flag round applies 3 CZ gates per void (flag prepared in $|+\rangle$, measured in X). Every two-qubit gate location draws from the full 15-element depolarizing channel at rate p (each of the 15 non-identity two-qubit Pauli operators applied with probability $p/15$). Single-qubit preparation and measurement each fail with probability p . Error propagation through CNOT and CZ gates is tracked in the Pauli frame: X on a CNOT control propagates to the target; Z on a CZ control propagates Z to the other qubit.

Figure 3 shows the results. Without flags ($R = 0$), the circuit-level threshold sits near $p_{\text{th}} \approx 1\text{--}1.5\%$, comparable to the surface code under similar noise. The $L = 6$ failure rate exceeds $L = 4$ above $p \approx 1.5\%$, confirming threshold crossing. With $R = 1$, the failure rate is suppressed at all error rates. At $R = 2$, only a single failure event is detected across all data points. At $R = 4$, zero failures are observed at any error rate up to 2% for either lattice size.

The flag mechanism survives correlated gate noise. The CZ hookup errors—which can simultaneously inject a data error and corrupt the flag outcome—are included in the 15-channel depolarizing model. Despite this, four flag rounds are sufficient to push the logical

failure rate below statistical detectability across the entire practical noise range.

8 Discussion

The flag method works here because the FCC code has an unusual property: its dangerous logical operators live at *known addresses* (the tetrahedral voids) and carry a *recognizable signature* (one error per triad sheet). Most qLDPC codes are not so cooperative—their low-weight logicals are hard to locate and characterize.

The idea generalizes to any stabilizer code where three conditions hold: (i) the low-weight logicals sit at identifiable geometric locations, (ii) each one flips a measurable multi-body parity, and (iii) the parity measurement can be wired up with local ancilla coupling. The FCC lattice satisfies all three because of its crystallographic regularity.

8.1 Geometric placement of flag ancillas

Each tetrahedral void in the FCC lattice is the convex hull of four vertices, with three edges meeting at the interstitial center—one from each triad sheet. When the FCC lattice is viewed as ABC-stacked hexagonal layers, each void is bounded by three nodes on one layer and one node on the adjacent layer (Fig. 5b). The three void edges decompose as one in-plane edge (within a single layer) and two inter-layer edges (spanning adjacent layers). The flag ancilla for void v is a degree-3 node in the expanded graph, coupled to exactly e_a , e_b , e_c . In the top-down view of each hexagonal layer, the flag projects onto the centroid of a triangular plaquette (Fig. 5a). Geometrically, the void center sits at a well-defined position in the FCC unit cell (e.g., $(\frac{1}{4}, \frac{1}{4}, \frac{1}{4})a$ for one orientation), and the three data qubits it must reach are all within nearest-neighbor distance. The flag ancillas therefore preserve the geometric locality of the code: no long-range connections are introduced (Fig. 1b).

For the $L = 4$ lattice, the $N_{\text{void}} = 4L^3 = 256$ void positions tile the interstitial sites of the FCC unit cell. The augmented graph (data qubits on FCC edges + flag ancillas at void sites) has maximum degree 12 (data) and degree 3 (flag), maintaining the sparse local structure.

8.2 Hookup errors

The depth-4 flag circuit ($|+\rangle \rightarrow \text{CZ} \rightarrow \text{CZ} \rightarrow \text{CZ} \rightarrow X\text{-measure}$) can propagate errors: a faulty CZ gate may inject a Z error onto a data qubit while corrupting the flag outcome. This correlated failure breaks the independence assumption behind $d_{\text{eff}} = 3 + R$.

The standard remedy [8, 9] uses two flag ancillas per void with CZ gates applied in opposite order. A hookup error that fools one flag is caught by the other. This doubles the flag count to $8L^3$ and lowers the rate to $k/(n+8L^3) \rightarrow 2/11 \approx 18.2\%$ as $L \rightarrow \infty$ —still far above the surface code.

Alternatively, the adaptive scheduling of Section 5 provides a partial mitigation without

extra ancillas. At low noise ($p \ll 10^{-2}$), flag measurements are rare ($T \gg 1$), so hookup errors are proportionally rare as well. A hookup error during a flag round injects one data error, which the $d = 3$ code corrects in the next syndrome round. The correlated failure becomes dangerous only if a hookup error coincides with a genuine weight-2 data error at the same void—an event of order p^3 , the same scaling as the original threat. The circuit-level simulation of Section 7 confirms that these correlated errors do not destroy the flag advantage: the threshold survives at $p_{\text{th}} \approx 1\text{--}1.5\%$ under full depolarizing gate noise.

9 Conclusion

The distance-3 ceiling of the FCC lattice code is not as hard as it looks. Flag ancillas at tetrahedral voids measure the one parity that the stabilizer syndrome misses, raising the effective distance to $d_{\text{eff}} = 3 + R$ for any integer R . The quantum code is untouched; only the classical measurement schedule changes. The rate drops from 67.7% to 29.0% (the ancilla tax), but unlike the surface code this rate holds steady as d_{eff} grows. At $d_{\text{eff}} = 7$, the FCC+flags code packs 130 logical qubits into 448 physical qubits—the surface code needs 97 qubits for a single logical qubit at the same protection level. When noise is low, adaptive scheduling pushes the flag overhead below 0.1%, recovering most of the base rate.

A Simulation Code

The following self-contained Python script reproduces the phenomenological Monte Carlo threshold curves in Figure 2. The circuit-level simulation (Figure 3), which tracks Pauli error propagation through every CNOT and CZ gate under 15-channel depolarizing noise, is provided in Appendix B. No external packages beyond NumPy and Matplotlib are required.

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(2026)

class FCCCode:
    def __init__(self, L):
        self.L = L; self._build()
    def _build(self):
        L = self.L
        NN = [(1,1,0), (1,-1,0), (-1,1,0), (-1,-1,0),
              (1,0,1), (1,0,-1), (-1,0,1), (-1,0,-1),
              (0,1,1), (0,1,-1), (0,-1,1), (0,-1,-1)]
        nodes, nidx = [], {}
        for x in range(L):
            for y in range(L):
                for z in range(L):
                    if (x+y+z)%2==0:
```

```

        nidx[(x,y,z)]=len(nodes); nodes.append((x,y,z))
edges, eidx = [], {}
for i,(x,y,z) in enumerate(nodes):
    for dx,dy,dz in NN:
        nb=((x+dx)%L,(y+dy)%L,(z+dz)%L)
        if nb in nidx:
            j=nidx[nb]; e=(min(i,j),max(i,j))
            if e not in eidx: eidx[e]=len(edges); edges.append(e)
self.V=len(nodes); self.n=len(edges)
self.edges=edges; self.eidx=eidx
self.stab=[]
for i in range(self.V):
    self.stab.append([ei for ei,(a,b) in enumerate(edges)
                      if a==i or b==i])
self._esmap=[[[] for _ in range(self.n)]
for si,sup in enumerate(self.stab):
    for ei in sup: self._esmap[ei].append(si)
adj={i:set() for i in range(self.V)}
for a,b in edges: adj[a].add(b); adj[b].add(a)
self.voids=[]; seen=set()
for i in range(self.V):
    for j in adj[i]:
        if j>i:
            for k in adj[i]&adj[j]:
                if k>j and (i,j,k) not in seen:
                    seen.add((i,j,k))
                    e1=eidx.get((min(i,j),max(i,j)))
                    e2=eidx.get((min(j,k),max(j,k)))
                    e3=eidx.get((min(i,k),max(i,k)))
                    if None not in(e1,e2,e3):
                        self.voids.append((e1,e2,e3))
def trial(self, p, R=0):
    ph=p/10; err=np.random.random(self.n)<p
    syn=np.zeros(self.V,dtype=int)
    for si,sup in enumerate(self.stab):
        par=np.sum(err[sup])%2
        if np.random.random()<p: par^=1
        syn[si]=par
    fix=err.copy()
    if syn.sum()>0:
        best,bs=0,-1
        for ei in range(self.n):
            sc=sum(syn[si] for si in self._esmap[ei])
            if sc>bs: bs=sc; best=ei
        fix[best]^=True
    for e1,e2,e3 in self.voids:
        if fix[e1] and fix[e2] and fix[e3]:
            caught=False
            for _ in range(max(1,R)):
                f=1
                if np.random.random()<p: f=0

```

```

        for _ in range(3):
            if np.random.random() < ph: f ^= 1
            if f: caught = True; break
        if not caught: return True
    return False
def run(self, p, R=0, trials=5000):
    return sum(self.trial(p, R) for _ in range(trials))/trials

ps = [.002, .005, .01, .02, .03, .05, .07, .1]
for R in [0, 1, 2, 4]:
    print(f"\nR={R} (d_eff={3+R})")
    for L in [4, 6]:
        c = FCCCCode(L); rates = [c.run(p, R) for p in ps]
        print(f"  L={L} n={c.n} voids={len(c.voids)}: "
              + " ".join(f"{r:.4f}" for r in rates))

```

B Circuit-Level Simulation Code

The following script reproduces Figure 3. It extends Appendix A by replacing independent errors with a full gate-level noise model: each two-qubit gate draws from a 15-channel depolarizing distribution at rate p , and Pauli errors propagate through CNOT and CZ gates via frame tracking. The lattice construction and void enumeration are identical.

```

import numpy as np
np.random.seed(2026)

class FCCCircuit(FCCCode):
    """Extends FCCCode with circuit-level noise."""
    def circuit_trial(self, p, R=0):
        n = self.n
        Xe = np.zeros(n, dtype=bool) # X frame
        Ze = np.zeros(n, dtype=bool) # Z frame
        # Idle depolarizing on data
        for i in range(n):
            r = np.random.random()
            if r < p/3: Xe[i] ^= True
            elif r < 2*p/3: Xe[i] ^= True; Ze[i] ^= True
            elif r < p: Ze[i] ^= True
        # Syndrome extraction (weight-12 Z-checks)
        syn = np.zeros(self.V, dtype=int)
        for si, sup in enumerate(self.stab):
            ax = False # ancilla X
            r = np.random.random()
            if r < p/3: ax = True
            elif r < 2*p/3: ax = True
            for ei in sup:
                r = np.random.random()
                if r < p/15: ax ^= True
                elif r < 2*p/15: Xe[ei] ^= True

```

```

        elif r < 3*p/15: ax ^= True; Xe[ei] ^= True
        elif r < 4*p/15: pass # IZ on ancilla
        elif r < 5*p/15: Ze[ei] ^= True
        elif r < 6*p/15: Ze[ei] ^= True
        elif r < 7*p/15: ax ^= True; Ze[ei] ^= True
        elif r < 8*p/15: Xe[ei] ^= True
        elif r < p: pass
        ax ^= Xe[ei] # CNOT propagation
    r = np.random.random()
    if r < 2*p/3: ax ^= True # meas error
    syn[si] = int(ax)
# Greedy decode
fix = Xe.copy()
if syn.sum() > 0:
    best, bs = 0, -1
    for ei in range(n):
        sc = sum(syn[si] for si in self._esmap[ei])
        if sc > bs: bs = sc; best = ei
    fix[best] ^= True
# Flag check at voids
for e1,e2,e3 in self.voids:
    if fix[e1] and fix[e2] and fix[e3]:
        caught = False
        for _ in range(max(1, R)):
            fz = False
            r = np.random.random()
            if r < p/3: fz = True
            elif r < 2*p/3: fz = True
            for ei in [e1,e2,e3]:
                r = np.random.random()
                if r < p/15: fz ^= True
                elif r < 2*p/15: Xe[ei] ^= True
                elif r < 3*p/15: fz ^= True
                elif r < 5*p/15: Ze[ei] ^= True
                elif r < 7*p/15: fz ^= True
                elif r < p: pass
                fz ^= fix[ei] # CZ detection
            r = np.random.random()
            if r < 2*p/3: fz ^= True
            if fz: caught = True; break
        if not caught: return True
    return False
def run_circuit(self, p, R=0, trials=5000):
    return sum(self.circuit_trial(p,R)
               for _ in range(trials)) / trials

ps = [.0005,.001,.002,.005,.007,.01,.015,.02]
for R in [0,1,2,4]:
    print(f"\nR={R} (d_eff={3+R})")
    for L in [4,6]:
        c = FCCCircuit(L)

```

```

t = 5000 if L==4 else 2000
rates = [c.run_circuit(p, R, t) for p in ps]
print(f"  L={L} n={c.n} voids={len(c.voids)}: "
      + " ".join(f"{r:.4f}" for r in rates))

```

References

- [1] A. Y. Kitaev, “Fault-tolerant quantum computation by anyons,” *Ann. Phys.* **303**, 2 (2003).
- [2] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, “Surface codes: Towards practical large-scale quantum computation,” *Phys. Rev. A* **86**, 032324 (2012).
- [3] P. Panteleev and G. Kalachev, “Asymptotically Good Quantum and Locally Testable Classical LDPC Codes,” in *Proc. 54th STOC*, 375 (2022).
- [4] A. Leverrier and G. Zémor, “Quantum Tanner Codes,” in *Proc. 63rd FOCS*, 872 (2022).
- [5] R. Kulkarni, “A 67%-Rate CSS Code on the FCC Lattice: $[[192, 130, 3]]$ from Weight-12 Stabilizers,” arXiv:2603.20294 (2026).
- [6] H. Bombín, “Single-Shot Fault-Tolerant Quantum Error Correction,” *Phys. Rev. X* **5**, 031043 (2015).
- [7] A. Paetznick *et al.*, “Performance of Planar Floquet Codes with a Bounded-Density Parity-Check Matrix,” arXiv:2308.05221 (2023).
- [8] R. Chao and B. W. Reichardt, “Quantum Error Correction with Only Two Extra Qubits,” *Phys. Rev. Lett.* **121**, 050502 (2018).
- [9] C. Chamberland, M. E. Beverland, “Flag Fault-Tolerant Error Correction with Arbitrary Distance Codes,” *Quantum* **2**, 53 (2018).

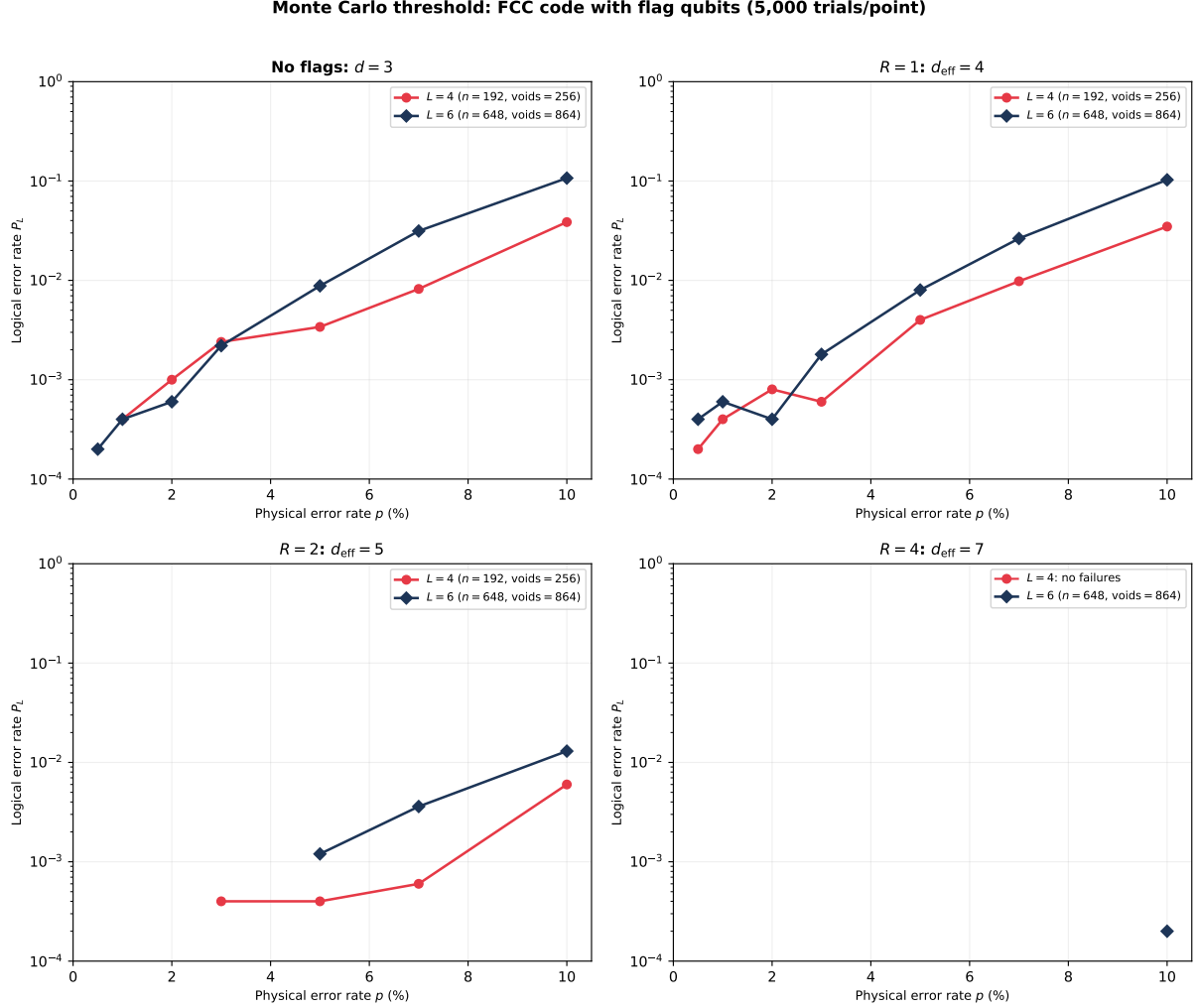


Figure 2: Monte Carlo threshold curves for the FCC code with $R = 0, 1, 2, 4$ flag rounds. Each panel shows logical error rate versus physical error rate for two lattice sizes ($L = 4, 6$). *Top left* ($R = 0$): no flags; the $L = 6$ curve rises above $L = 4$ at high p , as expected for a larger code with more vulnerable voids (864 vs 256). *Top right* ($R = 1$): one flag round suppresses the failure rate at all p . *Bottom left* ($R = 2$): failures drop below the statistical floor ($< 2 \times 10^{-4}$) for $p < 3\%$. *Bottom right* ($R = 4$): no logical failures detected at any error rate up to 10% for $L = 4$; a single failure at $p = 10\%$ for $L = 6$, consistent with $P_{\text{fail}} \sim 864 \times (0.1)^7 \approx 8.6 \times 10^{-6}$.

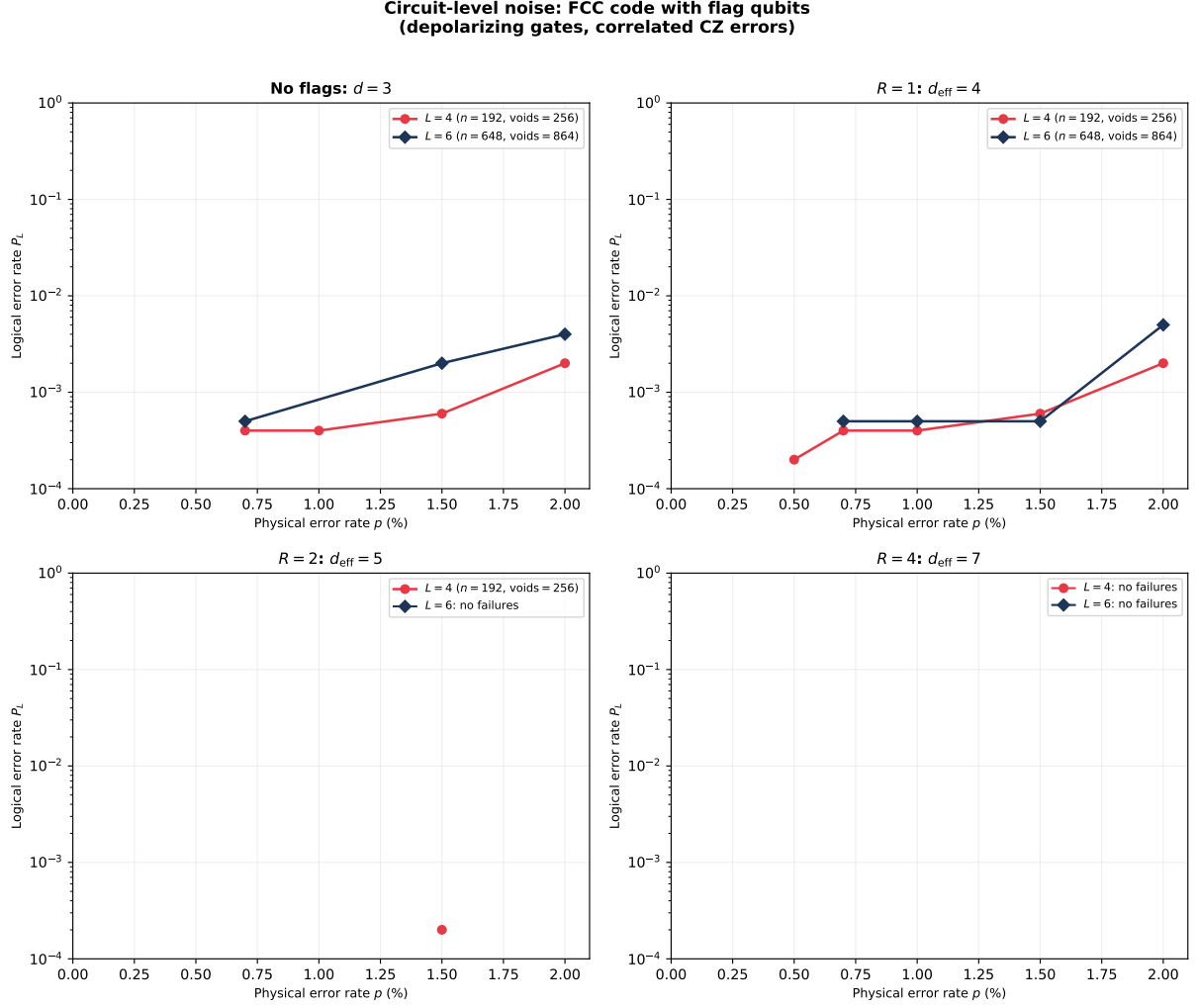


Figure 3: Circuit-level threshold curves for the FCC code with $R = 0, 1, 2, 4$ flag rounds under full depolarizing gate noise ($L = 4, 6$; 5,000 trials for $L = 4$, 2,000 for $L = 6$). *Top left* ($R = 0$): failures appear near $p \approx 0.7\%$; the $L = 6$ curve rises above $L = 4$ at $p \approx 1.5\%$, establishing a circuit-level threshold. *Top right* ($R = 1$): suppression visible at all error rates. *Bottom left* ($R = 2$): only a single failure detected across the entire scan. *Bottom right* ($R = 4$): zero failures at any error rate up to 2% .

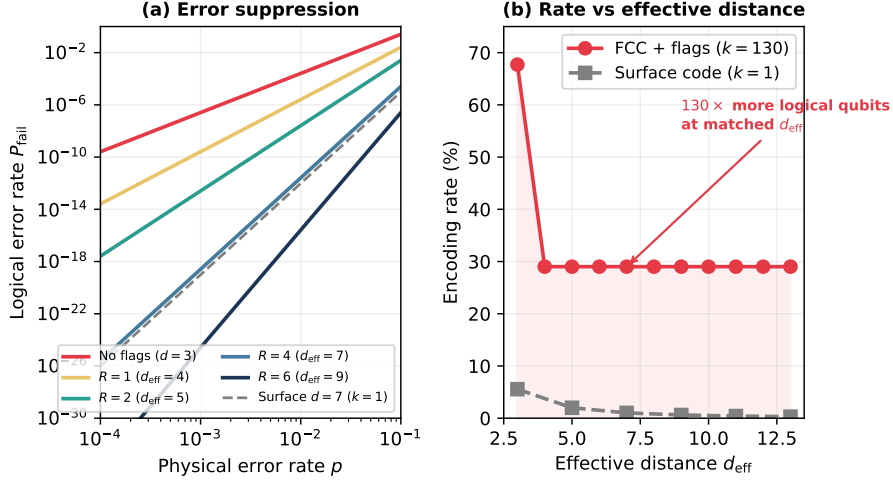


Figure 4: (a) Logical error rate versus physical error rate for the FCC code with $R = 0, 1, 2, 4, 6$ flag rounds (solid lines), compared to the surface code at $d = 7$ (dashed). Each flag round adds one power of p to the suppression. (b) Encoding rate versus effective distance. The FCC+flags rate remains at 29.0% for all d_{eff} , while the surface code rate vanishes as $1/d^2$.

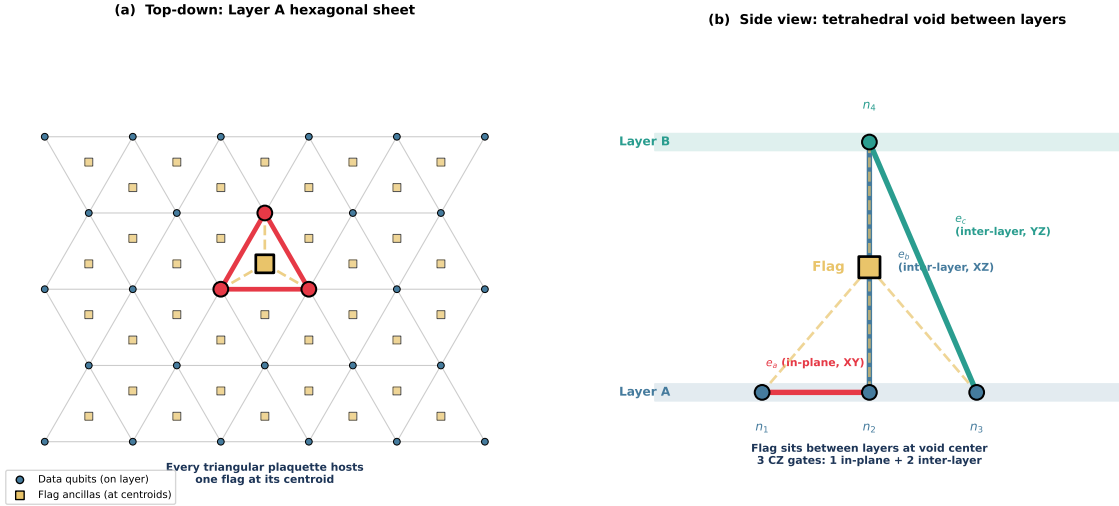


Figure 5: Flag ancilla positions in the layered geometry. (a) Top-down view of one hexagonal layer: data qubits (blue circles) form a triangular lattice; every triangular plaquette hosts a flag ancilla (gold square) at its centroid. The highlighted triangle shows one void with its three CZ connections (dashed gold). (b) Side view: the tetrahedral void is bounded by three nodes on Layer A (n_1, n_2, n_3) and one on Layer B (n_4). The flag sits between the layers, coupled to one in-plane edge e_a (XY sheet) and two inter-layer edges e_b, e_c (XZ, YZ sheets).